

CS 630: Software Testing and Quality Assurance

2-2 Journal: Improving Bug Communication in QA Teams

Malgorzata Debska

Southern New Hampshire University

Dr. Ben Torrales

July 14, 2026

Journal: Improving Bug Reporting and Communication

Throughout my coursework and programming assignments, I have realized that identifying a software defect is only one part of a QA analyst's responsibility. Just as important is documenting the defect in a clear and organized way so that developers and other team members can easily understand, reproduce, and fix the issue. During several class projects, I noticed that incomplete or unclear bug reports often delayed the debugging process because developers had to spend additional time asking for missing information instead of working on a solution. These experiences have shown me that effective communication is just as important as technical testing skills. A well-written bug report helps the entire team collaborate more efficiently, reduces misunderstandings, and leads to faster resolution of software defects.

One observation I have made is that ineffective bug reports usually lack enough information to reproduce the issue. For example, a report that simply states, "The application crashes," provides very little value because it does not explain what the user was doing when the crash occurred. Without reproduction steps, developers may not be able to recreate the defect, making it difficult to identify the root cause. I have also seen situations during coursework where classmates described a problem but did not include screenshots, error messages, or information about the environment in which the bug occurred. These missing details often lead to confusion and unnecessary back-and-forth communication.

There are several common challenges that contribute to incomplete or unclear bug reports in real-world software projects. One challenge is inconsistent documentation. Bugs may be reported through emails, chat messages, verbal conversations, or handwritten notes instead of a

centralized tracking system. As a result, important information can easily be forgotten or lost. Another challenge is the lack of standardized reporting practices. Different testers may describe the same issue in different ways, making it difficult for developers to understand the problem quickly. Incorrect severity or priority ratings are another common issue. For example, a cosmetic problem such as a misspelled word should not receive the same priority as a defect that prevents users from logging into the application. Finally, time pressure can cause testers to submit reports before collecting all the necessary information, resulting in incomplete documentation that delays the debugging process (Roman, 2023).

Using consistent language is one of the best ways to improve communication between QA analysts and developers. When every team member uses standardized software testing terminology, there is less room for misunderstanding. Terms such as defect, severity, priority, expected result, actual result, and environment have specific meanings that help everyone interpret bug reports consistently. The ISTQB Glossary provides standard definitions that are widely recognized throughout the software testing industry (International Software Testing Qualifications Board [ISTQB], n.d.). Consistent terminology also helps product owners, testers, developers, and project managers communicate more effectively because everyone shares the same understanding of the reported issue. This reduces confusion during defect triage meetings and allows the team to prioritize work more accurately.

A standard bug report should include several structural components that make the report complete and easy to understand. First, every defect should have a unique bug ID so it can be tracked throughout its lifecycle. A clear and descriptive title should summarize the issue in a few words. The report should also identify the testing environment, including the operating system, browser, application version, or device where the problem occurred. One of the most important

sections is the list of steps needed to reproduce the defect because developers must be able to recreate the problem before they can fix it. The report should also include the expected result, describing what should have happened, and the actual result, explaining what happened instead. Assigning both severity and priority helps the team understand the technical impact of the defect and determine how quickly it should be addressed. Finally, supporting evidence such as screenshots, videos, application logs, or error messages provides additional context that often makes diagnosing the problem much easier. Together, these structural components create a complete record that improves communication, reduces misunderstandings, and helps developers resolve issues more efficiently.

If I were leading a QA team, I would recommend creating a standardized bug-reporting process using Jira. Every defect would be submitted through the same template, requiring testers to complete mandatory fields before a report could be created. These fields would include a descriptive summary, environment information, reproduction steps, expected and actual results, severity, priority, and supporting attachments. I would also encourage peer reviews of bug reports before they are assigned to developers, especially high-priority issues. In addition, I believe short training sessions on writing effective bug reports would improve documentation quality across the team. Standardized templates combined with consistent training would reduce incomplete reports, improve collaboration between QA and development teams, and help defects move through the development process more efficiently.

Overall, this module reinforced that successful software testing depends on more than identifying defects. Clear documentation and professional communication are essential parts of quality assurance because they allow developers to reproduce problems, understand their impact, and implement solutions more quickly. As I continue developing my software engineering skills, I

recognize that writing complete, organized, and consistent bug reports will be just as important as performing the testing itself. Strong communication ultimately leads to better teamwork, faster defect resolution, and higher-quality software.

References

International Software Testing Qualifications Board. (n.d.). *ISTQB glossary*.

<https://glossary.istqb.org/>

Roman, A. (2023). *Software testing strategies: A testing guide for the 2020s*. Mercury Learning and Information.